CMSC132: Algorithm Complexity

Chapter 6

By Naren Ayinala

May 2026

Contents

1 What Algorithm Complexity Actually Measures **3**

1.1 Big O, Big Omega, and Big Theta 3

2 Growth Rates: Pictures and Intuition **3**

2.1 Graph 1: Constant, logarithmic, linear, and quadratic 3

2.2 Graph 2: Comparing larger growth rates 4

3 Calculating Complexity from Java Code **4**

3.1 One loop: $O(n)$ 4

3.2 Two independent consecutive loops: still $O(n)$ 4

3.3 Two nested loops: $O(n^2)$ 4

3.4 Triangular nested loops: still $O(n^2)$ 5

3.5 Repeated doubling or halving: $O(\log n)$ 5

3.6 Outer linear, inner logarithmic: $O(n \log n)$ 5

3.7 Logarithmic outer, linear inner: also $O(n \log n)$ 5

3.8 Not all nested loops multiply to n^2 5

3.9 A deceptive $O(n)$ nested loop 6

3.10 Three nested loops: $O(n^3)$ 6

3.11 Conditionals, recursion, and space 6

4 Searching Algorithms: Watch the Array Change (or Not!) **6**

4.1 Linear search: unsorted data is fine 7

4.2 Binary search: array MUST be sorted 7

4.3 Jump search: sorted array, block by block 8

4.4 Search comparison 8

5 Sorting Properties: What the Words Really Mean **8**

5.1 In-place sorting . . . 8

5.2 Stable sorting . . . 8

5.3 Internal versus external sorting . . . 9

5.4 Adaptive sorting . . . 9

5.5 Property comparison . . . 9

5.6 Bubble sort: adjacent swaps, one pass at a time . . . 9

5.7 Selection sort: select the minimum . . . 10

5.8 Insertion sort: grow a sorted prefix . . . 10

5.9 Merge sort: split, sort, merge . . . 11

5.10 Quick sort: partition around a pivot . . . 11

5.11 Heap sort: build a max heap, remove maxima . . . 12

6 Sorting Algorithm Summary Table **13**

7 Major Data Structures: Big O Reference **13**

7.1 Linear structures . . . 13

7.2 Stacks, queues, and deques . . . 14

7.3 Hash structures, trees, and heaps . . . 14

7.4 Graphs: common representations and traversals . . . 14

7.5 Sorting, searching, and structure choice: what to remember . . . 14

8 Algorithm Design Strategies (Short Review) **14**

9 Practice Questions with Worked Answers **15**

10 Final Exam Checklist **15**

1 What Algorithm Complexity Actually Measures

Imagine two programs solving the same problem. One takes 5 milliseconds for 100 elements; the other takes 10 milliseconds. Which is better for a million elements? We cannot answer from those two measurements alone! We need to understand how work *grows* with input size n. Complexity describes that scaling, not an exact stopwatch prediction. We generally count basic operations and focus on large n.

Time complexity describes the number of operations as input grows. **Auxiliary space complexity** describes extra memory beyond the input (and, depending on the convention, beyond the output). A function can take $O(n^2)$ time but $O(1)$ extra space. Conversely, copying an array takes $O(n)$ time and $O(n)$ extra space.

1.1 Big O, Big Omega, and Big Theta

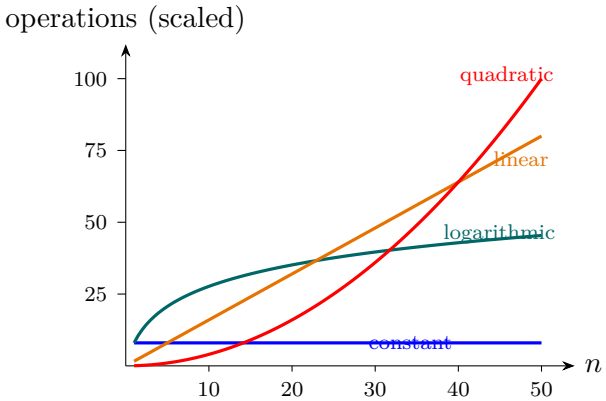
$O(f(n))$ is an asymptotic *upper* bound; $\Omega(f(n))$ is a lower bound; $\Theta(f(n))$ is a tight bound. For example $T(n) = 3n^2+10n+7$ is $\Theta(n^2)$, because the quadratic term eventually dominates. Big O is *not* a synonym for worst case: we can state a Big O bound for best-case or average-case time too. Always say which case you mean.

Simplifying: discard constant multipliers and lower-order terms. Thus $5n + 100 = O(n)$, $4n^2 + 6n = O(n^2)$, and $n \log_2 n + 3n = O(n \log n)$. The base of a logarithm does not matter in Big O because changing bases multiplies by a constant.

2 Growth Rates: Pictures and Intuition

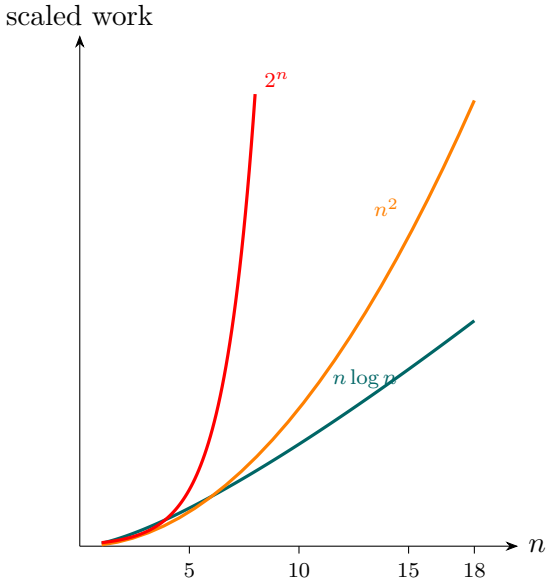
Growth	Common source	At $n = 1024$ (roughly)
$O(1)$	index into an array	1
$O(\log_2 n)$	binary search	10
$O(n)$	scan once	1,024
$O(n \log_2 n)$	efficient comparison sorting	10,240
$O(n^2)$	all pairs / nested loops	1,048,576
$O(2^n)$	enumerate all subsets	astronomically large

2.1 Graph 1: Constant, logarithmic, linear, and quadratic



Note: The curves use different scaling constants to make all four visible; compare their *shapes*, not their exact heights.

2.2 Graph 2: Comparing larger growth rates



Exponential growth quickly becomes unmanageable. However, $O(n^2)$ is not always slower than $O(n \log n)$ for *tiny* inputs: implementation constants and cache behavior matter.

3 Calculating Complexity from Java Code

3.1 One loop: $O(n)$

```
int total = 0; // O(1)
for (int i = 0; i < n; i++) { // n iterations
    total += i; // O(1) per iteration
}
// T(n) = c*n + d => O(n)
```

For $n = 5$, the body runs 5 times. For $n = 1000$, 1000 times. This is linear growth.

3.2 Two independent consecutive loops: still $O(n)$

```
for (int i = 0; i < n; i++) work();
for (int j = 0; j < n; j++) work();
// n + n = 2n => O(n), NOT O(n^2)
```

3.3 Two nested loops: $O(n^2)$

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        work();
    }
}
// n * n = n^2 => O(n^2)
```

If $n = 4$, `work()` runs $4 \cdot 4 = 16$ times. If $n = 10$, it runs 100 times.

3.4 Triangular nested loops: still $O(n^2)$

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j <= i; j++) work();
}
```

For $n = 4$, the inner body executes $1 + 2 + 3 + 4 = 10$ times. In general $1 + 2 + \cdots + n = n(n + 1)/2$, so the dominant term is $n^2/2$ and the bound is $\Theta(n^2)$.

3.5 Repeated doubling or halving: $O(\log n)$

```
for (int i = 1; i < n; i *= 2) work();
// i = 1, 2, 4, 8, 16, ...
while (n > 1) { n /= 2; work(); }
```

After k doublings, $i = 2^k$. We stop near $2^k = n$, meaning $k \approx \log_2 n$. For $n = 32$, only about 5 iterations! (Exact count depends on the loop boundary.)

3.6 Outer linear, inner logarithmic: $O(n \log n)$

```
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j *= 2) work();
}
// n outer iterations * log(n) inner iterations
```

3.7 Logarithmic outer, linear inner: also $O(n \log n)$

```
for (int i = n; i > 0; i /= 2) {
    for (int j = 0; j < n; j++) work();
}
```

3.8 Not all nested loops multiply to n^2

```
int j = 0;
for (int i = 0; i < n; i++) {
    while (j < i + 1) j++;
}
// j never resets: at most n total increments => O(n)
```

This is an example of *amortized counting*: instead of multiplying loop labels, count the total number of times the inner body actually executes.

3.9 A deceptive $O(n)$ nested loop

```
for (int i = 1; i < n; i *= 2) {
    for (int j = 0; j < i; j++) work();
}
// 1 + 2 + 4 + ... < 2n => O(n)
```

Even though there are two loops, the work forms a geometric series, not $n \log n$.

3.10 Three nested loops: $O(n^3)$

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        for (int k = 0; k < n; k++) work();
```

3.11 Conditionals, recursion, and space

```
if (flag) {
    for (int i = 0; i < n; i++) work();
} else {
    for (int i = 0; i < n*n; i++) work();
}
// Worst-case O(n^2); best branch O(n)
```

A recursion that makes *one* call on $n-1$ each time is often $O(n)$ time and $O(n)$ stack space. A balanced divide-and-conquer recurrence $T(n) = 2T(n/2) + O(n)$ is $O(n \log n)$, as in merge sort. Two calls on $n-1$ without memoization can lead to exponential time.

Pattern	Counting idea	Time
One full scan	n	$O(n)$
Two consecutive scans	$2n$	$O(n)$
Two full nested scans	n^2	$O(n^2)$
Triangle	$n(n+1)/2$	$O(n^2)$
Doubling loop	$\log_2 n$	$O(\log n)$
Scan + doubling inside	$n \log_2 n$	$O(n \log n)$
Geometric nested work	$1 + 2 + 4 + \cdots + n$	$O(n)$
Three full nested scans	n^3	$O(n^3)$

4 Searching Algorithms: Watch the Array Change (or Not!)

Searching usually *does not modify* the array. Instead, the set of candidate indices shrinks. The examples below explicitly show which elements are checked.

4.1 Linear search: unsorted data is fine

Search for 7 in $[9, 3, 12, 7, 5]$. We check each item from left to right:

Step	Array (unchanged)	Action
1	$[\mathbf{9}, 3, 12, 7, 5]$	$9 \neq 7$, continue
2	$[9, \mathbf{3}, 12, 7, 5]$	$3 \neq 7$, continue
3	$[9, 3, \mathbf{12}, 7, 5]$	$12 \neq 7$, continue
4	$[9, 3, 12, \mathbf{7}, 5]$	Found at index 3

```
static int linearSearch(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i;
    }
    return -1;
}
```

Best case $O(1)$ (first element), worst case $O(n)$ (last or absent), auxiliary space $O(1)$.

4.2 Binary search: array MUST be sorted

Search for 23 in $[2, 5, 8, 12, 16, 23, 38, 56, 72]$:

Step	low	high	mid / value	Next action
1	0	8	4 / 16	$23 > 16$: discard indices 0–4
2	5	8	6 / 38	$23 < 38$: discard indices 6–8
3	5	5	5 / 23	Found at index 5

$$[2, 5, 8, 12, \boxed{16}, 23, 38, 56, 72]$$

$$[\cancel{2}, \cancel{5}, \cancel{8}, \cancel{12}, \cancel{16}, 23, \boxed{38}, 56, 72]$$

$$[2, 5, 8, 12, 16, \boxed{23}, 38, 56, 72]$$

Crossed-out elements were eliminated after step 1; the array itself remains unchanged.

```
static int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Best case $O(1)$; worst case $O(\log n)$; iterative extra space $O(1)$. Binary search on an unsorted array is **incorrect**, even though the loop would still execute logarithmically.

4.3 Jump search: sorted array, block by block

Consider $[1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31]$, target 21. With block size 4, check block ends at indices 3 (7), 7 (15), and 11 (23). Since $21 \leq 23$, scan indices 8 (17), 9 (19), and 10 (21): found. The array never changes. With an optimal jump size around $\sqrt{n}$, worst-case time is $O(\sqrt{n})$ and extra space is $O(1)$.

4.4 Search comparison

Method	Sorted input?	Best time	Worst time
Linear search	No	$O(1)$	$O(n)$
Binary search (array)	Yes	$O(1)$	$O(\log n)$
Jump search	Yes	$O(1)$	$O(\sqrt{n})$
Hash table lookup (typical)	No	$O(1)$	$O(n)$
Balanced BST lookup	Maintains ordering	$O(1)$	$O(\log n)$

Hash-table $O(1)$ is an *expected/average* bound under suitable hashing and load conditions, not a universal guarantee.

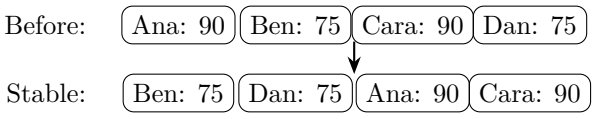
5 Sorting Properties: What the Words Really Mean

5.1 In-place sorting

An **in-place** algorithm rearranges the original data without requiring an additional array of comparable size. A few counters or a temporary variable are fine. A conventional bubble sort uses one temporary variable to swap adjacent entries and therefore uses $O(1)$ auxiliary space: **bubble sort is in-place**. Standard merge sort for arrays uses a separate temporary array of length n, so it is **not in-place** under the usual definition. Recursive quicksort partitions in the original array, but its recursion stack uses extra memory; textbooks commonly call it in-place because it does not allocate an $O(n)$ merge buffer. Always distinguish array storage from recursion-stack storage.

5.2 Stable sorting

A stable sort preserves the *relative order* of records with equal sort keys. Imagine these students are sorted by score only:



Ben remains before Dan, and Ana remains before Cara. A sort that reversed either equal-score pair would be *unstable*. Bubble sort is stable when it swaps only on `a[j] > a[j+1]`, not when it swaps equal values unnecessarily.

5.3 Internal versus external sorting

Internal sorting happens when the working data fits in main memory (RAM). **External sorting** is designed for datasets too large to fit in RAM, so it uses disk or other external storage and manages I/O carefully. External merge sort commonly sorts chunks in memory and merges them from disk. This distinction is about the memory/storage setting, not whether the Java method is public or private.

5.4 Adaptive sorting

An **adaptive** sort uses existing order in the input to reduce work. Insertion sort is adaptive: if the array is already sorted, its inner shifting loop does almost nothing, giving $O(n)$ time. An optimized bubble sort with a `swapped` flag can also stop after one pass on an already sorted array. Standard selection sort remains $O(n^2)$ even when already sorted because it keeps searching for the minimum.

5.5 Property comparison

Algorithm	In-place?	Stable?	Adaptive?
Bubble (optimized)	Yes	Yes	Yes
Selection (standard)	Yes	No	No
Insertion	Yes	Yes	Yes
Merge (standard arrays)	No	Yes	Typically no
Quick (standard partition)	Usually called yes	No	Typically no
Heap	Yes	No	No

Important: Stability and adaptivity can depend on the exact implementation. "Yes" for merge sort assumes stable merging on ties.

5.6 Bubble sort: adjacent swaps, one pass at a time

Compare neighboring values, swapping when the left one is larger. The largest unsorted value moves right on each pass. Here is the array *after every complete pass*:

Step	Array state	Explanation
Start	$[5, 2, 4, 1, 3]$	Original unsorted array
Pass 1	$[2, 4, 1, 3, 5]$	Largest remaining value bubbles to index 4
Pass 2	$[2, 1, 3, 4, 5]$	Largest remaining value bubbles to index 3
Pass 3	$[1, 2, 3, 4, 5]$	Largest remaining value bubbles to index 2
Pass 4	$[1, 2, 3, 4, 5]$	Largest remaining value bubbles to index 1

```
static void bubbleSort(int[] a) {
    for (int end = a.length - 1; end > 0; end--) {
        boolean swapped = false;
        for (int j = 0; j < end; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted
```

```
    }
}
```

Optimized bubble sort: best $O(n)$, average/worst $O(n^2)$, extra space $O(1)$, stable if equal values are not swapped.

5.7 Selection sort: select the minimum

At each pass, scan the unsorted suffix to find its minimum and put that value at the next position. The prefix grows sorted:

Step	Array state	Explanation
Start	$[5, 2, 4, 1, 3]$	Original array
Pass 1	$[1, 2, 4, 5, 3]$	Find minimum 1 in remaining suffix; swap into index 0
Pass 2	$[1, 2, 4, 5, 3]$	Find minimum 2 in remaining suffix; swap into index 1
Pass 3	$[1, 2, 3, 5, 4]$	Find minimum 3 in remaining suffix; swap into index 2
Pass 4	$[1, 2, 3, 4, 5]$	Find minimum 4 in remaining suffix; swap into index 3

```
static void selectionSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < a.length; j++)
            if (a[j] < a[min]) min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
    }
}
```

Best/average/worst $O(n^2)$: even if sorted, it scans the remaining suffix. Extra space $O(1)$. Standard selection sort is not stable because a long-distance swap can reorder equal keys.

5.8 Insertion sort: grow a sorted prefix

Think about sorting cards in your hand. Take the next value and slide it left until it fits. After each step, the prefix through the inserted value is sorted:

Step	Array state	Explanation
Start	$[5, 2, 4, 1, 3]$	First element is a sorted prefix
Insert 2	$[2, 5, 4, 1, 3]$	Shift larger prefix values right, insert 2
Insert 4	$[2, 4, 5, 1, 3]$	Shift larger prefix values right, insert 4
Insert 1	$[1, 2, 4, 5, 3]$	Shift larger prefix values right, insert 1
Insert 3	$[1, 2, 3, 4, 5]$	Shift larger prefix values right, insert 3

```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
```

```
            a[j + 1] = a[j]; j--;
        }
        a[j + 1] = key;
    }
}
```

Best $O(n)$ for already sorted input, average/worst $O(n^2)$; extra space $O(1)$; stable. Very useful for short or nearly sorted arrays.

5.9 Merge sort: split, sort, merge

Merge sort uses divide and conquer. Splitting does not sort anything; the work occurs when sorted subarrays are merged. Consider $[5, 2, 4, 1, 3]$:

Phase	Array groups
Split 1	$[5, 2]$ $[4, 1, 3]$
Split 2	$[5]$ $[2]$ $[4]$ $[1, 3]$
Split 3	$[5]$ $[2]$ $[4]$ $[1]$ $[3]$
Merge small groups	$[2, 5]$ $[4]$ $[1, 3]$
Merge right group	$[2, 5]$ $[1, 3, 4]$
Final merge	$[1, 2, 3, 4, 5]$

When merging $[2, 5]$ and $[1, 3, 4]$, compare the front values: choose 1, then 2, then 3, then 4, then 5. Each merge costs linear work in its combined length. There are $O(\log n)$ levels, so total time is $O(n \log n)$ in best, average, and worst cases. Standard array merge sort uses $O(n)$ extra buffer space.

```
static void mergeSort(int[] a) {
    mergeSort(a, new int[a.length], 0, a.length - 1);
}
static void mergeSort(int[] a, int[] temp, int lo, int hi) {
    if (lo >= hi) return;
    int mid = lo + (hi - lo) / 2;
    mergeSort(a, temp, lo, mid);
    mergeSort(a, temp, mid + 1, hi);
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi)
        temp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
    while (i <= mid) temp[k++] = a[i++];
    while (j <= hi) temp[k++] = a[j++];
    for (int t = lo; t <= hi; t++) a[t] = temp[t];
}
```

The `<=` tie rule takes the left equal-key record first, preserving stability.

5.10 Quick sort: partition around a pivot

For a transparent trace, use Lomuto partitioning with the *last* element as pivot. Start with $[5, 2, 4, 1, 3]$ and pivot 3:

Action	Array
Start, pivot 3	$[5, 2, 4, 1, 3]$
See $5 > 3$, no swap	$[5, 2, 4, 1, 3]$
See $2 \leq 3$, swap with first large element	$[2, 5, 4, 1, 3]$
See $4 > 3$, no swap	$[2, 5, 4, 1, 3]$
See $1 \leq 3$, move left	$[2, 1, 4, 5, 3]$
Put pivot into final position	$[2, 1, 3, 5, 4]$
Sort left $[2, 1]$	$[1, 2, 3, 5, 4]$
Sort right $[5, 4]$	$[1, 2, 3, 4, 5]$

Now 3 is in its final sorted index, and we recursively sort each side. Average time $O(n \log n)$, worst $O(n^2)$ (e.g. repeatedly bad pivots), typical stack space $O(\log n)$ but worst $O(n)$. Standard quicksort is not stable.

```
static void quickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}
static int partition(int[] a, int lo, int hi) {
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++) {
        if (a[j] <= pivot) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++;
        }
    }
    int t = a[i]; a[i] = a[hi]; a[hi] = t;
    return i;
}
```

5.11 Heap sort: build a max heap, remove maxima

Use a max heap so the largest value can be moved to the end. Starting array $[5, 2, 4, 1, 3]$:

Action	Array (vertical bar separates heap/sorted suffix)
Build max heap	$[5, 3, 4, 1, 2]$
Swap root with last, sift down	$[4, 3, 2, 1 \mid 5]$
Swap root with last heap entry, sift	$[3, 1, 2 \mid 4, 5]$
Swap and sift	$[2, 1 \mid 3, 4, 5]$
Swap final two	$[1 \mid 2, 3, 4, 5]$

Heap sort takes $O(n \log n)$ worst-case time and $O(1)$ auxiliary array space; it is not stable. Building the initial heap bottom-up costs $O(n)$, and the n root removals cost $O(\log n)$ each.

```
static void heapSort(int[] a) {
    int n = a.length;
    for (int i = n / 2 - 1; i >= 0; i--) siftDown(a, i, n);
    for (int end = n - 1; end > 0; end--) {
        int t = a[0]; a[0] = a[end]; a[end] = t;
        siftDown(a, 0, end);
    }
}
```

```
static void siftDown(int[] a, int i, int size) {
    while (2*i + 1 < size) {
        int child = 2*i + 1;
        if (child + 1 < size && a[child + 1] > a[child]) child++;
        if (a[i] >= a[child]) break;
        int t = a[i]; a[i] = a[child]; a[child] = t;
        i = child;
    }
}
```

6 Sorting Algorithm Summary Table

Algorithm	Best	Average	Worst	Extra space	Stable?
Bubble (early stop)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge (arrays)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick (typical)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ avg. stack	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Why not always use merge sort? It needs an auxiliary array in a conventional array implementation. **Why not always use quicksort?** It is usually fast in practice but has quadratic worst-case behavior without safeguards. **Why use insertion sort?** It is simple and often excellent for very small or nearly sorted arrays.

7 Major Data Structures: Big O Reference

This section is meant to be your quick revision table. Operations refer to conventional implementations and are not all guaranteed worst-case bounds. For example, a hash table has expected $O(1)$ lookup under good hashing, while a balanced BST has worst-case $O(\log n)$ lookup.

7.1 Linear structures

Structure	Access index	Search	Insert front	Insert end	Delete front
Array (fixed size)	$O(1)$	$O(n)$	$O(n)$	$O(1)$ if room	$O(n)$
ArrayList	$O(1)$	$O(n)$	$O(n)$	amort. $O(1)$	$O(n)$
Singly linked list	$O(n)$	$O(n)$	$O(1)$	$O(1)$ with tail	$O(1)$
Doubly linked list	$O(n)$	$O(n)$	$O(1)$	$O(1)$ with tail	$O(1)$

For a linked list, *inserting or deleting at a known node* can be $O(1)$ with the required references (deleting an arbitrary node in a singly linked list generally requires access to its predecessor). Finding that node first may cost $O(n)$. ArrayList end insertion is *amortized* $O(1)$ because occasional capacity growth copies all elements.

7.2 Stacks, queues, and deques

Structure	Add	Remove	Peek
Stack (array or linked)	push $O(1)$ amort./worst*	pop $O(1)$	top $O(1)$
Queue (circular array/linked)	enqueue $O(1)$ amort./worst*	dequeue $O(1)$	front $O(1)$
Deque (ArrayDeque)	add first/last amort. $O(1)$	remove ends $O(1)$	peek ends $O(1)$

* Linked implementations with suitable references give worst-case $O(1)$; dynamically resized arrays give amortized $O(1)$ for insertion.

7.3 Hash structures, trees, and heaps

Structure	Search/contains	Insert	Delete	Find min/max
HashMap / HashSet (typical)	expected $O(1)$	expected $O(1)$	expected $O(1)$	$O(n)$ scan
Unbalanced BST (worst)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Balanced BST (AVL / red-black)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Binary min-heap	$O(n)$ arbitrary key	$O(\log n)$	$O(\log n)$ root	min $O(1)$
Binary max-heap	$O(n)$ arbitrary key	$O(\log n)$	$O(\log n)$ root	max $O(1)$

Hash-table worst-case operations can degrade to $O(n)$ in conventional collision-handling models; some Java HashMap collision bins may become balanced trees, giving improved bounds under appropriate conditions. A heap is excellent for *getting the minimum or maximum*, but not for arbitrary-key lookup.

7.4 Graphs: common representations and traversals

Let V be the number of vertices and E the number of edges.

Operation	Adjacency list	Adjacency matrix
Storage	$O(V + E)$	$O(V^2)$
Test edge $u \to v$	$O(\deg(u))$	$O(1)$
List neighbors of u	$O(\deg(u))$	$O(V)$
BFS / DFS full traversal	$O(V + E)$	$O(V^2)$

7.5 Sorting, searching, and structure choice: what to remember

If you need indexed random access, prefer an array or ArrayList. If you need fast membership without order, a hash set is a strong default. If you need *sorted* keys and logarithmic updates, use a balanced search tree. If you repeatedly need the smallest or largest priority, use a heap. If you need to process items in arrival order, use a queue. The operation you perform most often should guide your choice.

8 Algorithm Design Strategies (Short Review)

Brute force: try possibilities directly. **Divide and conquer:** split, solve, combine (merge sort). **Greedy:** make a locally appealing choice (Kruskal's algorithm). **Dynamic programming:** reuse answers to overlapping subproblems. **Backtracking:** explore and undo decisions. **Branch and bound:**

eliminate branches using proven bounds. These are ways to *design* algorithms, not complexity categories by themselves.

9 Practice Questions with Worked Answers

1. What is the complexity of a loop running n times followed by two nested loops of n iterations each?
Answer: $n + n^2 = \Theta(n^2)$; the quadratic term dominates.

2. What if the inner loop doubles j on each iteration, and the outer loop runs n times?
Answer: $n \cdot \log_2 n = \Theta(n \log n)$.

3. Why is binary search invalid on $[10, 2, 8, 5]$?
Answer: The direction in which binary search discards half the array relies on sorted order; without it, the target might be in the discarded half.

4. Is bubble sort in-place? Stable?
Answer: Yes, standard bubble sort uses constant extra memory. Yes, when swapping only *strictly* out-of-order adjacent items, it preserves equal-key relative order.

5. Why does selection sort still take $O(n^2)$ when the input is already sorted?
Answer: It still scans each unsorted suffix to locate the minimum, performing roughly $n(n-1)/2$ comparisons.

6. Can a doubly linked list access its middle element in $O(1)$?
Answer: Not by index. It still must walk from an end, taking $O(n)$ in general. It can remove a *known* node in $O(1)$.

7. Which data structure supports expected $O(1)$ membership testing, and which supports worst-case $O(\log n)$ sorted membership testing?
Answer: HashSet for expected constant-time membership; a balanced search tree such as TreeSet for logarithmic operations.

8. Does $O(n \log n)$ always beat $O(n^2)$ for $n = 3$?
Answer: No. Asymptotic complexity concerns growth, not exact runtimes for small input sizes; constants and overhead can reverse the comparison.

10 Final Exam Checklist

Be able to: identify the input size; count iterations rather than guessing from loop nesting; distinguish $O(n)$, $O(\log n)$, $O(n \log n)$, and $O(n^2)$; trace sorting passes by hand; explain *why* a sort is stable or in-place; state whether a search requires sorted input; distinguish expected, amortized, and worst-case bounds; and choose a data structure based on its dominant operations.